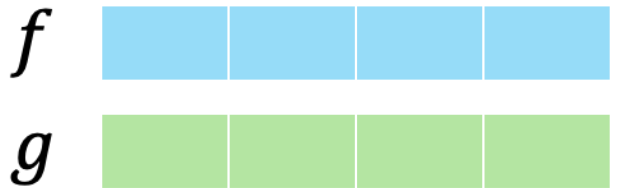
Almost Linear-Time Permutation Checks

Benedikt Bünz[1], **Jessica Chen[1]**, Zachary DeStefano[1], Binyi Chen[2]

1.New York University, 2.Tsinghua University

The Setting of PermCheck



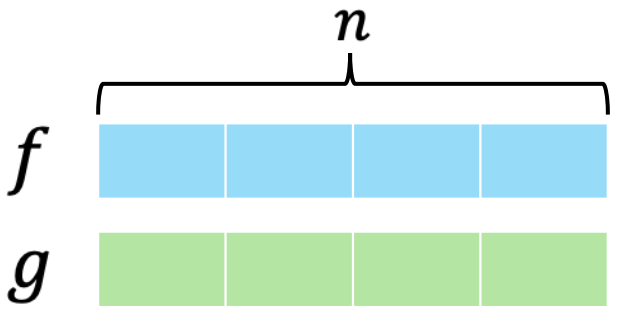
Prover

"f, g are permutations of each other"

Verifier

- public-coin
- little comp. power

The Setting of PermCheck: a related question



Prover

"f, g are permutations of each other w.r.t. σ"

Verifier

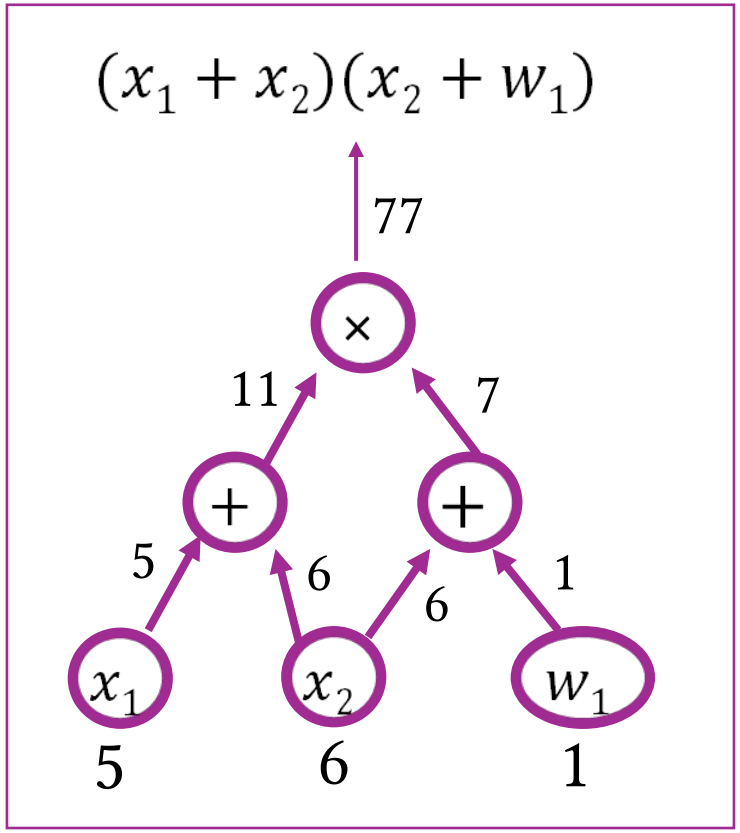
?

- public-coin
- little comp. power

Reasons to care about PermCheck

- Wiring check in circuits

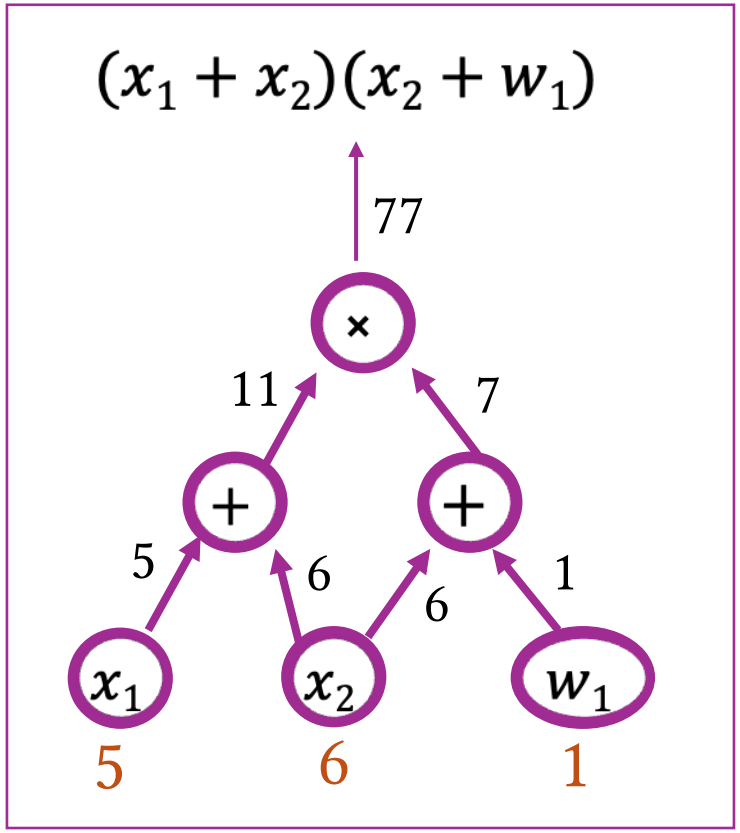




Reasons to care about PermCheck

- Wiring check in circuits

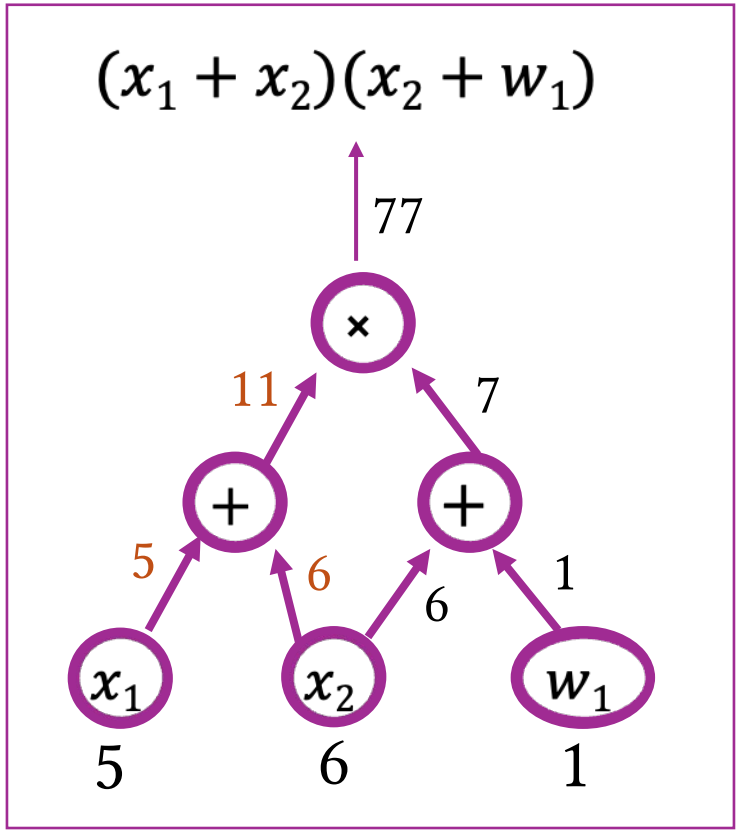
5	6	1									



Reasons to care about PermCheck

- Wiring check in circuits

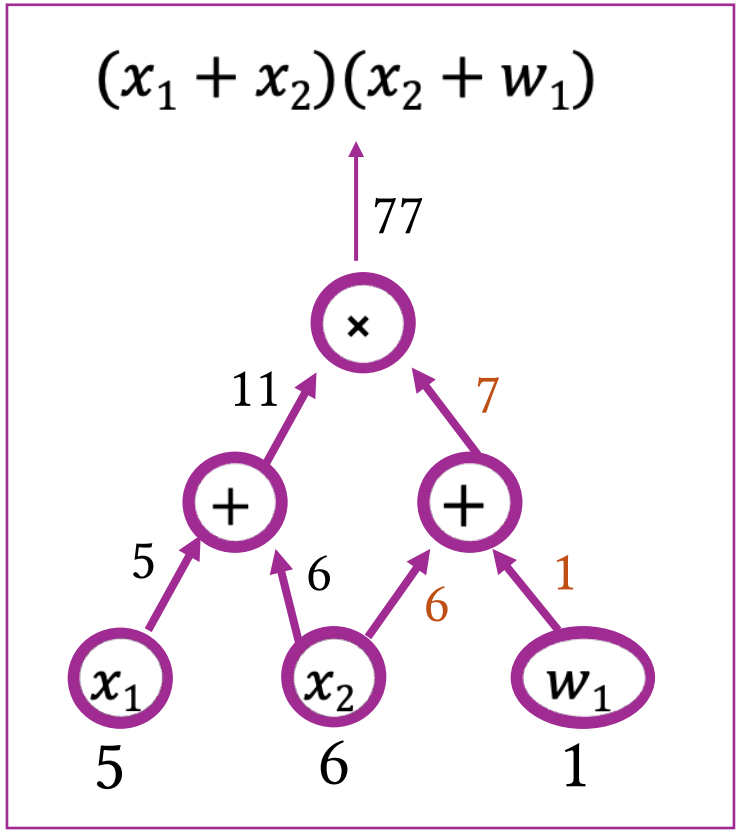
5	6	1	5	6	11						



Reasons to care about PermCheck

- Wiring check in circuits

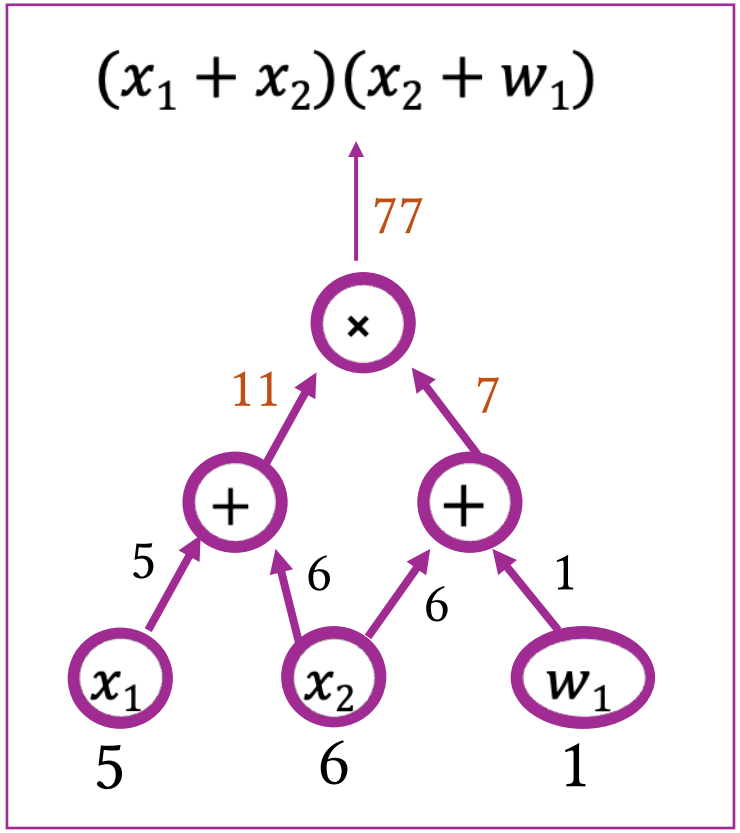
5	6	1	5	6	11	6	1	7			



Reasons to care about PermCheck

- Wiring check in circuits

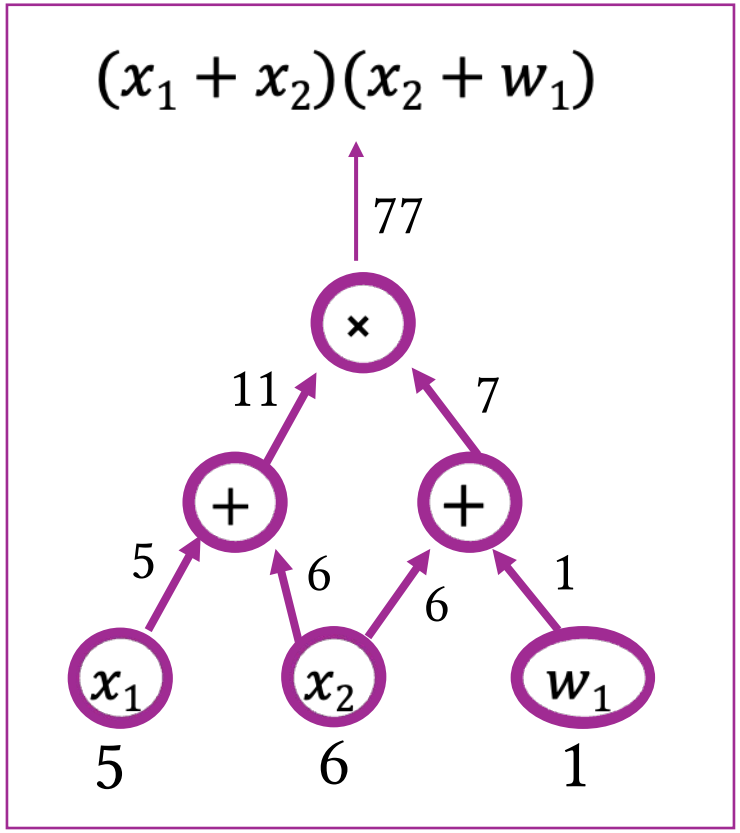
5	6	1	5	6	11	6	1	7	11	7	77



Reasons to care about PermCheck

- Wiring check in circuits

5	6	1	5	6	11	6	1	7	11	7	77

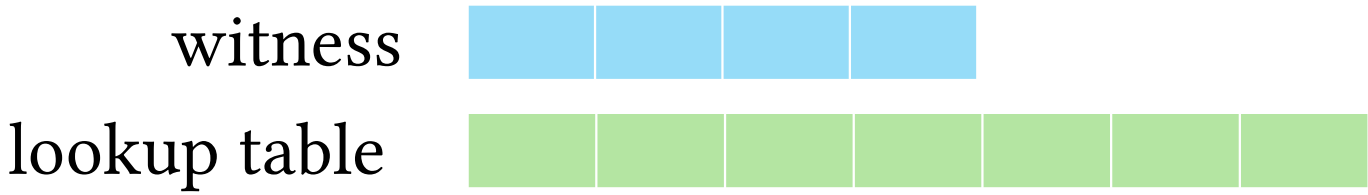


Reasons to care about PermCheck

- Wiring check in circuits
- Memory checking
  - Offline memory-checking [BEG+91]:
    - read before write, write after read
    - PermCheck between $(init, W)$ and $(R, final)$

Reasons to care about PermCheck

- Wiring check in circuits
- Memory checking
- Highly related to lookup arguments



"every element in the witness can be found in the table"

PermCheck (on the Boolean Hypercube)

For f, g: $\{0,1\}^{\log n} \to \mathbb{F}$,

$\exists\ \sigma: \{0,1\}^{\log n} \to \{0,1\}^{\log n}$ s.t.

$$f(x) = g(\sigma(x)) \quad \forall x \in \{0,1\}^{\log n}$$

Previous Approach

- Grand product check: for random $r \leftarrow \mathbb{F}$,

$$\prod_{x \in \{0,1\}^{\log n}} (r + f(x)) = \prod_{x \in \{0,1\}^{\log n}} (r + g(x))$$

  - $O(n)$ field-size commitments.
  - Soundness error $\frac{n}{|\mathbb{F}|}$.

	Grand product	Grand product with GKR	Our result
Commitment Cost	$n\,\lambda$ bits	$\|w\|$	$\|w\|$ + PCS prover
Soundness Error	$n/\|\mathbb{F}\|$	$(n+\log^2 n)/\|\mathbb{F}\|$	$\text{polylog}(n)\,/\|\mathbb{F}\|$
Prover Cost (# $\mathbb{F}$ ops)	$O(n)$	$O(n)$	BiPerm $O(n)$ MulPerm $n \cdot \tilde{O}(\sqrt{\log n})$
Verifier Cost	$\log n$	$\log^2 n$	$\log n$

	Grand product	Grand product with GKR	Our result
Commitment Cost	$n\,\lambda$ bits	$\lvert w\rvert$	$\lvert w\rvert$ + F
Soundness Error	$n/\lvert\mathbb{F}\rvert$	$(n+\log^2 n)/\lvert\mathbb{F}\rvert$	polylog(n) $/\lvert\mathbb{F}\rvert$
Prover Cost (# $\mathbb{F}$ ops)	$O(n)$	$O(n)$	BiPerm $O(n)$ MulPerm $n \cdot \tilde{O}(\sqrt{\log n})$
Verifier Cost	$\log n$	$\log^2 n$	$\log n$

limited PCS choice

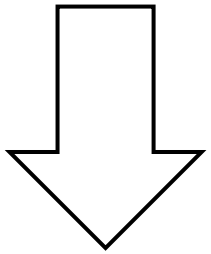
Any PCS!

Reduce PermCheck to Sumcheck

Define: $\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$

$2\log n$ variables

$$f(x) = g(\sigma(x)) \quad \forall x \in \{0,1\}^{\log n}$$



$$\sum_{x \in \{0,1\}^{\log n}} f(x) \cdot \mathbf{1}_\sigma(x,y) = g(y) \quad \forall y \in \{0,1\}^{\log n}$$

$$\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$$

Reduce PermCheck to Sumcheck

$$f(x) = g(\sigma(x)) \quad \forall x \in \{0,1\}^{\log n}$$

Arithmetize s.t. this is multilinear in y

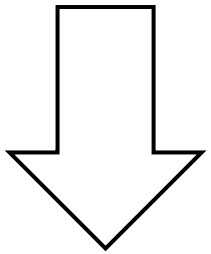
$$\sum_{x \in \{0,1\}^{\log n}} f(x) \cdot \mathbf{1}_\sigma(x,y) = g(y) \quad \forall y \in \{0,1\}^{\log n}$$

$$\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$$

Reduce PermCheck to Sumcheck

$$f(x) = g(\sigma(x)) \quad \forall x \in \{0,1\}^{\log n}$$

$\tilde{\mathbf{1}}_\sigma(x,y)$ is multilinear in y



$$\sum_{x \in \{0,1\}^{\log n}} f(x) \cdot \tilde{\mathbf{1}}_\sigma(x,y) = g(y) \quad \forall y \in \{0,1\}^{\log n}$$

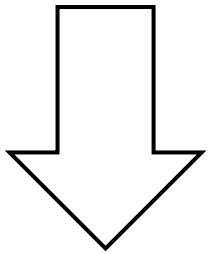
Total degree in y is $\log n$ on both sides

$$\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$$

Reduce PermCheck to Sumcheck

$$f(x) = g(\sigma(x)) \quad \forall x \in \{0,1\}^{\log n}$$

$\tilde{\mathbf{1}}_\sigma(x,y)$ is multilinear in y



$$\sum_{x \in \{0,1\}^{\log n}} f(x) \cdot \tilde{\mathbf{1}}_\sigma(x,y) = g(y) \quad \forall y \in \{0,1\}^{\log n}$$

$$\sum_{x \in \{0,1\}^{\log n}} f(x) \cdot \tilde{\mathbf{1}}_\sigma(x,\alpha) = g(\alpha) \quad \alpha \in \mathbb{F}^{\log n}$$

$$\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$$

How to arithmetize $\tilde{\mathbf{1}}_\sigma(\mathbf{x},\mathbf{y})$?

- $2\log n$ variables.
- Needs to be multilinear in y.
- As the representation of the permutation map σ, it will be committed to and opened using a PCS.
- Want linear prover time.

$$\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$$

How to arithmetize $\tilde{\mathbf{1}}_\sigma(\mathbf{x},\mathbf{y})$?



A product of bit-wise equalities?

$$\prod_{i=1}^{\log n} \mathrm{eq}(\sigma_i(x), y_i)$$

Can we do better?

- Degree $\log n$ in every x_i
- Sumcheck cost $O(n \log^2 n)$ field mults

$$\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$$

How to arithmetize $\tilde{\mathbf{1}}_\sigma(\mathbf{x},\mathbf{y})$?



Can we just use a multilinear extension, which is degree 1 in every x_i?

We can't.

- n^2 evaluation points, n nonzero.
- Even with the best PCS we know, takes $O(n\lambda)$ time to open.

$$\mathbf{1}_\sigma(X,Y) = \begin{cases} 1 & \sigma(X) = Y \\ 0 & \sigma(X) \neq Y \end{cases}$$

How we arithmetize $\tilde{\mathbf{1}}_\sigma(\mathbf{x},\mathbf{y})$

BiPerm: degree *2* in every x_i

- One sumcheck. Prover cost: $O(n)$ $\mathbb{F}$.
- Needs to preprocess large but sparse polynomials.
- Requires a PCS whose cost solely depends on the sparsity of the polynomial.

MulPerm: degree $\sqrt{\log n}$ in every x_i

- Two sumchecks. Prover cost: $n * \tilde{O}\left(\sqrt{\log n}\right)$ $\mathbb{F}$.
- Any PCS!

Open Questions

- PermCheck for arbitrary Boolean circuits with linear prover time?
  - Solved in follow-up work, with constant soundness error

- PermCheck for arbitrary Boolean circuits with linear prover time AND negligible soundness error?

References

[AST24] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In Annual International Conference on the Theory and Applications of Cryptographic Techniques, pages 3–33. Springer, 2024.

[BayGro12] Bayer, S., Groth, J. (2012). Efficient Zero-Knowledge Argument for Correctness of a Shuffle. In: Pointcheval, D., Johansson, T. (eds) Advances in Cryptology – EUROCRYPT 2012. EUROCRYPT 2012. Lecture Notes in Computer Science, vol 7237. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-29011-4_17

[BC24] Benedikt Bünz and Jessica Chen. Proofs for Deep Thought: Accumulation for large memories and deterministic computations/ Cryptology ePrint Archive, Report 2024/325. https://eprint.iacr.org/2024/325. 2024.

[BEG+91] Manuel Blum, William S. Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the correctness of memories. In 32nd FOCS, pages 90–99. IEEE Computer Society Press, October 1991.

[CBBZ22] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. Cryptology ePrint Archive, Report 2022/1355. https://eprint.iacr.org/2022/1355. 2022.

[Cha81] D. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. Commun. ACM, 24(2):84–88, 1981.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530. https:// eprint.iacr.org/2022/1530. 2022.

[KRS25] Dmitry Khovratovich and Ron D. Rothblum and Lev Soukhanov. How to Prove False Statements: Practical Attacks on Fiat-Shamir. Cryptology ePrint Archive, Report 2025/118. https://eprint.iacr.org/2025/118. 2025.

[Neff01] C. Andrew Neff. A verifiable secret shuffle and its application to e-voting. In: Proceedings of the 8th ACM conference on Computer and Communications Security, 2001, 116–25.

[Set19] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. Cryptology ePrint Archive, Report 2019/550. https://eprint.iacr.org/2019/550. 2019.

[SL20] Srinath Setty and Jonathan Lee. Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275. https://eprint.iacr.org/2020/1275. 2020.

[ST25] Srinath Setty and Justin Thaler. Twist and Shout: Faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Report 2025/105. https://eprint.iacr.org/2025/105

[STW23] Srinath Setty, Justin Thaler, and Riad S. Wahby. Lasso: Unlocking the lookup singularity, 2023.